



Amrita School of AI
COIMBATORE CAMPUS

23AID304 · HIGH-PERFORMANCE AND CLOUD COMPUTING

Getting Started

Everything you need in week one. Where the course material lives, how to get it, how to work on the coding exercises, and how to submit them.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

COURSE

23AID304 High-Performance and Cloud Computing

REPOSITORY

github.com/Amrita-School-of-AI/hpc-23aid304

READ THIS

In week one, before the first lab

TIME NEEDED

About 45 minutes, once

OFFERING

Odd Semester 2026-27

Read this once, in week one

Everything for this course lives in one public GitHub repository. This guide takes you from a laptop with nothing set up to a working copy of the material, a compiler that runs the tests, and a submission you can be confident in.

Work through it in order. It takes about forty-five minutes and you never have to do it again.

BY THE END OF THIS GUIDE YOU WILL HAVE

- A GitHub account with an SSH key that works.
- The whole course repository on your machine.
- A compiler and MPI installed, and the public tests running.
- Your own fork, where every push automatically checks your work and shows you a results table.
- Submitted one exercise correctly.

Part 1: What is in the repository

github.com/Amrita-School-of-AI/hpc-23aid304

It is public. You do not need permission and you do not need to be added to anything.

```
hpc-23aid304/
├── handbook/      the course handbook: week plan, marks, policies, setup
├── notes/         five notes booklets, Units 0 to 4
├── slides/        the lecture decks
├── labs/          lab sheets, and the cluster setup guide
├── assignments/   the assignment briefs
├── mini-project/  the course project catalogue
├── exercises/     the coding exercises <- you work here
├── submissions/   where you put your answers
└── tools/check.py runs the public tests and prints a results table
```

FOLDER	WHAT IT IS	WHEN YOU NEED IT
handbook/	Start here. Week-by-week plan, how marks work, policies, what to install	Week 1
notes/	Self-contained booklets. You are not expected to buy or read a textbook	Throughout
slides/	What was on screen in class	After each lecture
labs/	The lab sheet for each session, with what to submit	Before each lab
assignments/	Four assignments, part of the 10 percent lab and assignment component	Weeks 3, 6, 9, 12
exercises/	Auto-graded coding exercises	Throughout
mini-project/	Twelve project options, 30 percent of the course	From week 6

NOTE · THE NOTES ARE THE TEXTBOOK

Units 0 to 4 are written to be read on their own. Unit 0 exists because the rest of the course assumes things that are easy to forget: cache lines, threads, and how to time a program honestly. Read it in week one even if it looks familiar.

Inside one exercise

```
exercises/ex06-openssl-reduction/  
├─ ex06-openssl-reduction.pdf  the problem statement. READ THIS FIRST  
├─ starter/reduction.c        the file you edit  
├─ include/reduction.h        the interface. Do not change it  
├─ tests/public.c             the tests you can run yourself  
└─ selfcheck.sh               compiles and runs those tests
```

Part 2: A GitHub account and an SSH key

You can download the material without any of this. You need it to use your own fork, which is where the automatic checking happens.

2.1 Create the account

Go to **github.com** and sign up. Use a username you are willing to show an employer: it becomes part of your professional record. Add your Amrita email so you can claim the free **GitHub Student Developer Pack** later.

2.2 Make an SSH key

An SSH key is a pair of files. The **private** key stays on your laptop forever. The **public** key you give to GitHub, which then recognises your machine without a password.

Linux, macOS, or Windows with WSL:

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press Enter three times to accept the default location. You may set a passphrase; if you do, you will be asked for it when you push.

Windows without WSL: open PowerShell and run the same command.

PITFALL · NEVER SHARE THE PRIVATE KEY

`~/.ssh/id_ed25519` is private. `~/.ssh/id_ed25519.pub` ends in `.pub` and is the one you share. If you ever paste a key into a chat, a website or a repository, delete it from GitHub and generate a new one. The same discipline applies to cloud credentials later in this course.

2.3 Give the public key to GitHub

Print it:

```
cat ~/.ssh/id_ed25519.pub
```

You get one line starting `ssh-ed25519 AAAA...`. Copy the whole line.

On GitHub: click your avatar, top right, then

Settings → SSH and GPG keys → New SSH key

Give it a title such as `laptop`, leave the type as *Authentication Key*, paste the line into the box, and click **Add SSH key**.

2.4 Check it works

```
ssh -T git@github.com
```

The first time it asks about authenticity: type `yes`. You should then see:

```
Hi <your-username>! You've successfully authenticated, but GitHub does not
provide shell access.
```

That message is a success, including the part about shell access.

PITFALL · IF IT SAYS PERMISSION DENIED (PUBLICKEY)

Three usual causes, in order of likelihood: you pasted the private key instead of the `.pub` one; you copied only part of the line; or the key is not being offered by the agent. For the last case:

```
bash eval "$(ssh-agent -s)" ssh-add ~/.ssh/id_ed25519
```

2.5 Tell git who you are

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Part 3: Get the material

3.1 Just reading it

If you only want the notes and slides, download the ZIP from the green **Code** button on the repository page. No account needed.

3.2 Working properly: fork, then clone

To get automatic checking you need **your own copy** on GitHub. That is a **fork**.

1. Open `github.com/Amrita-School-of-AI/hpc-23aid304`
2. Click **Fork**, top right, then **Create fork**
3. You now have `github.com/<your-username>/hpc-23aid304`

Clone **your fork**:

```
git clone git@github.com:<your-username>/hpc-23aid304.git
cd hpc-23aid304
```

Add the original as a second remote, so you can pull new material as I publish it:

```
git remote add upstream https://github.com/Amrita-School-of-AI/hpc-23aid304.git
```

3.3 Getting updates during the semester

I add material every week. To pull it into your fork:

```
git fetch upstream
git merge upstream/main
```

If that reports a conflict in a file you edited, it is almost always because you edited a `starter/` file in place. See Part 5 for the way to avoid that entirely.

Part 4: Install what you need

Everything is free.

Ubuntu, Debian, Linux Mint, or WSL:

```
sudo apt update
sudo apt install build-essential gdb git openmpi-bin libopenmpi-dev python3 python3-yaml
```

macOS:

```
xcode-select --install
brew install libomp open-mpi
```

Windows: install WSL2 with Ubuntu (`wsl --install` in an administrator PowerShell), then follow the Ubuntu instructions inside it. Do not fight native Windows for this course.

Check:

```
gcc --version
mpicc --version
echo | gcc -fopenmp -x c - -o /dev/null && echo "OpenMP works"
```

NOTE · CUDA IS NOT ON THIS LIST

The GPU exercises run on the School's cluster, not your laptop. Lab 2 shows you how to log in and how to ask the scheduler for a GPU. You do not need an NVIDIA card of your own.

Part 5: Working on an exercise

5.1 Read the statement

Open `exercises/<exercise>/<exercise>.pdf` first. It explains the problem, the traps, and exactly how the marks are allocated. The statements are written to be read; skipping them is the most common reason for a low mark on work that was nearly right.

5.2 Copy the starter, do not edit it in place

```
cd exercises/ex06-openmp-reduction
cp starter/reduction.c ../../submissions/ex06-openmp-reduction/AIE23001.c
```

Use **your own roll number** as the file name. Now edit that copy.

NOTE · WHY COPY RATHER THAN EDIT IN PLACE

Two reasons. The file you submit must be named for your roll number anyway, so you may as well create it now. And leaving `starter/` untouched means `git merge upstream/main` never conflicts when I publish new material.

5.3 Run the tests

```
./selfcheck.sh ../../submissions/ex06-openmp-reduction/AIE23001.c
```

You get one line per test:

```
PASS sum_small -- par_sum over 0..999 gives 499500
FAIL dot_ones -- par_dot of two all-ones arrays of length n is n
! tests/public.c:35 expected 4096 within 1e-12 relative, got 1024
SUMMARY 5/6 passed
```

The failure tells you the file, the line, what was expected, and what it got.

To check everything at once, from the top of the repository:

```
python3 tools/check.py
```

exercise	file	public tests	notes

ex06-openmp-reduction	AIE23001.c	PASS 6/6	
ex07-openmp-load-balance	workload.c	PASS 5/5	(starter)
ex11-mpi-halo-stencil	AIE23001.c	FAIL 1/3	ran, but the answer was wrong

Part 6: Automatic checking on every push

This is the part worth setting up.

Once you push to your fork, GitHub runs the public tests for you and shows a results table. You do not have to remember to run anything.

6.1 Turn Actions on, once

Forks have Actions disabled by default. In **your fork**:

Actions tab → I understand my workflows, go ahead and enable them

One click, once.

6.2 Push, and look

```
git add submissions/  
git commit -m "ex06: parallel reduction"  
git push
```

Open the **Actions** tab in your fork. The newest run shows a table:

EXERCISE	FILE	RESULT	PUBLIC TESTS	NOTES
ex06-omp-reduction	AIE23001.c	PASS	6/6	your submission
ex11-mpi-halo-stencil	AIE23001.c	FAIL	1/3	ran, but the answer was wrong

A green tick means every public test passed. A red cross means something did not, and the log shows exactly which case and why.

PITFALL · GREEN DOES NOT MEAN FULL MARKS

The workflow runs the **public** tests only. Your actual grade also includes:

- **hidden tests**, which exercise the same behaviour on inputs you have not seen
- a **determinism check**, which runs the tests repeatedly at 1, 2 and 4 threads and fails anything whose answer changes
- for some exercises, a **speedup measurement** against a serial reference

A parallel program with a data race very often passes a single run. That is precisely why the real grader runs things more than once, and why a green tick here is necessary but not sufficient.

CUDA exercises are skipped by the workflow, because GitHub's machines have no GPU. Run those on the cluster with `./selfcheck.sh`.

Part 7: Submitting

Submit **one file**, named for your roll number:

```
AIE23001.c
```

- No archive, no folder, no screenshot.
- No `main()` function unless the statement says otherwise.
- The extension matches the exercise: `.c`, `.cu` or `.yaml`.

I will tell you the collection channel for each deadline. Keeping your work in `submissions/` in your fork means the file you hand in is the same one the workflow has been testing all along.

What happens to your submission

Each file is graded in an isolated sandbox:

CHECK	WHAT IT DOES
Compiles cleanly	Built with <code>-Wall -Wextra -Werror</code>
Uses the required construct	The mechanism the exercise teaches is actually present
Public tests	The ones you already ran
Hidden tests	Same behaviour, unseen inputs
Determinism	Repeated runs at several thread counts must agree
Speedup	Parallel efficiency against a serial reference

You get back a **feedback PDF** showing every check, the marks, and for anything that failed, the exact assertion and the output.

Speedup is graded as **efficiency scaled to the machine's core count**, so the target adapts to whatever hardware the grading runs on. A two-core laptop and a 96-core cluster node are held to the same standard, not the same number.

Part 7b: How the marks work

COMPONENT	WEIGHT	WHAT IT IS
Continuous evaluation tests	30%	Three tests on Evolveus , around weeks 5, 10 and 14
Lab and class assignments	10%	Assignments 6%, coding exercises 2%, lab sheets 2%
Course project	30%	First review 10% around week 8, final review 20% in the last week
End - semester examination	30%	Three hours, all four units

There is **no separate mid-term**. The three continuous evaluation tests replace it.

PITFALL · THE CODING EXERCISES ARE WORTH 2 PERCENT, AND YOU SHOULD STILL DO ALL OF THEM

They are the cheapest possible way to find out that you do not understand something, and the continuous evaluation tests ask you to reason about exactly the things the exercises make you build. Every year some students skip the small components and are then surprised by a test on material they never practised.

The full breakdown, including what each project review is marked on, is in [handbook/course-handbook.pdf](#).

Part 8: Rules

Late work. Marked out of 80 percent within 48 hours; not accepted after that without documented cause. Tell me before the deadline, not after.

Collaboration. Discussing ideas, approaches and errors is encouraged and is how people actually learn. Submitting code you did not write is not. Every submission is compared against every other and against the reference, using a measure that ignores renamed variables and reformatting.

AI tools. You may use them to learn, to explain an error, or to explore an approach. You must be able to explain every line you submit. If you cannot, the work is not yours regardless of how it was produced.

Never commit a credential. Not in this course, not ever. A key pushed to a public repository is scanned and abused within minutes.

Part 9: When something goes wrong

Bring me, in this order:

1. What you expected to happen.
2. What actually happened.
3. The **exact** error text, copied as text.
4. The smallest program that still shows the problem.

Producing item 4 solves the problem about half the time on its own.

Things that catch nearly everyone

SYMPTOM	CAUSE
No speedup at all from OpenMP	You forgot <code>-fopenmp</code> . Every <code>#pragma omp</code> is silently ignored without it
<code>Permission denied (publickey)</code>	See Part 2.4
Merge conflict on <code>git merge upstream/main</code>	You edited a file in <code>starter/</code> . Work in <code>submissions/</code> instead
Actions tab shows nothing	You have not enabled workflows in your fork. See Part 6.1
Tests pass sometimes, fail other times	A data race. That is a real bug, not bad luck, and the grader will find it
<code>undefined reference to ...</code>	A linker error. A <code>.c</code> file or a library such as <code>-lm</code> is missing from the command

THE FIVE COMMANDS YOU WILL USE MOST

```
./selfcheck.sh` run the public tests for one exercise · `python3 tools/check.py` check everything at once · `git add .` && `git commit -m "...` && `git push` save and publish your work · `git fetch upstream` && `git merge upstream/main` pull new course material · `ssh -T git@github.com` confirm your key still works
```

Welcome to the course.

Dr. Abhijith Anandakrishnan Assistant Professor, Amrita School of AI